

Project Report

BSCS-2nd Semester

Fall 2025

Subject: Object Oriented Programming Lab

Date: 22 June 2026



Submitted to: Mohsin Shahzad

Submitted by:

Name	Hafsa Shahid Rida Sajid Areeba Fatima
Reg No	25-cs-181 25-cs-061 25-cs-221

GitHub link:

<https://github.com/AreebaFatima822/HITEC-OOP-SCMS-221-181-061>

Department of CS

HITEC University, Taxila

Introduction

Problem Statement

Modern educational institutions require an efficient system to manage academic and residential activities including students, faculty, courses, library resources, and hostel accommodation. Manual systems are slow, error-prone, and difficult to scale.

This project aims to develop a **Campus Management System** using Object-Oriented Programming (OOP) in C++ to automate and organize these operations efficiently.

System Goals

The system is designed to:

- Manage students, faculty, and staff using inheritance
- Handle course enrollment and capacity constraints
- Maintain a digital library catalog with books and journals
- Manage hostel room allocation and occupancy
- Generate reports and store data persistently
- Demonstrate advanced OOP concepts in a structured system

System Design (UML + Explanation)

UML Class Structure (Text Description)

- **Person (Abstract Class)**
 - Base class for Student, Faculty, Staff
 - Defines shared attributes (name, CNIC, age)
- **Student / Faculty / Staff**
 - Inherit from Person
 - Implement polymorphic behavior via `displayInfo()`
- **Course**
 - Associated with Faculty (aggregation)
 - Manages enrollment and capacity
- **Enrollment**
 - Links Student and Course (association)
- **LibraryItem (Abstract Class)**
 - Base class for Book and Journal
- **Book / Journal**
 - Inherit LibraryItem
 - Implement checkout functionality
- **Library**
 - Manages array of LibraryItem pointers (composition)
- **HostelManager**
 - Inherits Accommodation + Reportable (multiple inheritance)

- Manages HostelBlock
- **HostelBlock**
 - Composed of Room objects
- **Room**
 - Stores Student pointers (aggregation)

Relationships Summary

- Inheritance: Student, Faculty, Staff → Person
- Composition: HostelBlock → Room
- Aggregation: Course → Faculty
- Association: Enrollment → Student + Course
- Polymorphism: displayInfo(), checkout()
- Multiple Inheritance: HostelManager

OOP concepts implementation

1. Encapsulation

```
private:
    string name;
```

Used in all classes where data members are private.

2. Abstraction

```
class Person {
public:
    virtual void displayInfo() = 0;
};
```

Implemented using abstract classes:

3. Inheritance

```
class Student : public Person
```

Student inherits Person:

4. Polymorphism

```
void displayInfo() override;
```

Virtual function override:

5. Function Overriding

Child classes redefine base methods.

6. Constructors

```
Student(int id, string n);
```

Used for initializing objects:

7. Destructors

```
virtual ~Person() {}
```

Used for cleanup in base classes:

8. Pointers

```
Student* s;
```

Used for dynamic object handling:

9. Arrays of Objects

```
Room rooms[5];
```

10. Dynamic Memory

```
Room rooms[5];
```

11. Composition

HostelBlock contains Room objects.

12. Aggregation

Course holds Faculty pointer.

13. Association

Enrollment links Student and Course.

14. Multiple Inheritance

```
class HostelManager : public Accommodation, public Reportable
```

15. File Handling

```
ofstream fout("file.txt");
```

16. Exception Handling

Custom exceptions like OverdueException.

17. Friend Functions

Used for operator overloading.

18. Operator Overloading

```
bool operator==(Course c);
```

19. Static Members

Utility functions in Reports. Static members are shared across all objects and belong to the class instead of instances.

```
class Reports
{
public:
    static void printLine()
    {
        cout << "-----" << endl;
    }
};
```

20. Data Hiding

Private variables restrict direct access.

```
class Student
{
private:
    int studentID;    //This shows data is hidden

public:
    void setID(int id)
    {
        studentID = id;
    }
};
```

21. Constructor Overloading

Multiple constructors used in classes.

```
Book()
{
    copiesAvailable=0;
}
Book(int id, string t, string a, int year, string isbn, string g, int copies)
: libraryItem(id, t, a, year)
{
    ISBN=isbn;
    genre=g;
    copiesAvailable=copies;
}
```

22. Runtime Polymorphism

Virtual functions resolved at runtime.

23. Virtual Functions

Used in Person and LibraryItem.

```
class LibraryItem
{
public:
    virtual void checkout()
    {
```

24. Multilevel inheritance

Multilevel inheritance means a class is derived from another derived class.

25. Code Reusability

Base classes reused across modules.

Module Descriptions

Module 1 – Person Hierarchy

Implements inheritance and polymorphism for Student, Faculty, and Staff.

Module 2 – Course Management

Handles course enrollment, capacity checking, and operator overloading.

Module 3 – Library System

Manages books and journals using abstract classes and file handling.

Module 4 – Enrollment System

Links students with courses and manages grades.

Module 5 – Hostel Management

Handles room allocation using multiple inheritance and composition.

Module 6 – Reporting & Utilities

Generates reports, searches data, and saves hostel information to files.

GitHub Workflow

The project was developed using modular commits on GitHub.

Branch Strategy

- main branch: final stable code
- feature branches: each module developed separately

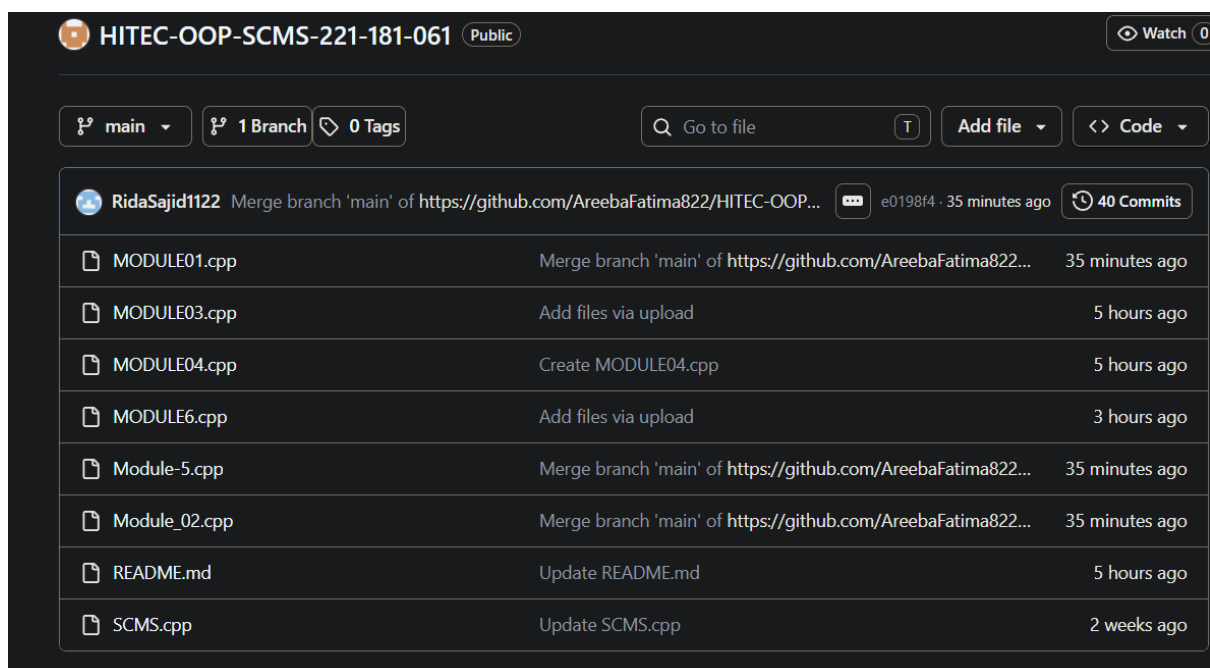
Commit Strategy

- Each module committed separately:

- Module 1: Person hierarchy
- Module 2: Course system
- Module 3: Library system
- Module 4: Enrolment system
- Module 5: Hostel system
- Module 6: Reports system

Benefits

- Easy debugging
- Organized development
- Clear version history



Challenges & Solutions

Challenge 1: Pointer Management

Handling Student pointers in rooms caused memory confusion.

Solution: Used clear ownership rules (Hostel does not delete students).

Challenge 2: File Handling Errors

Errors occurred while opening files using `ofstream`.

Solution: Ensured correct headers and used `.c_str()` where needed.

Challenge 3: Module Integration

Combining multiple modules caused dependency issues.

Solution: Structured code using headers and separated modules logically.

Challenge 4: Managing Multiple Modules Together

As the project grew into multiple modules (Person, Course, Library, Hostel, Reports), keeping track of interdependencies became difficult, especially when one module depended on classes from another.

Solution:

The system was structured into independent modules with clear responsibilities. Common classes were reused through proper inclusion instead of rewriting, and dependencies were minimized by using abstraction.

Challenge 5: Pointer and Memory Confusion

Using pointers for `Student`, `LibraryItem`, and `Room occupants` created confusion regarding ownership and lifetime of objects, leading to risks of memory misuse.

Solution:

A clear rule was followed where main objects (like `Student`) are created in `main()` and only referenced using pointers in other classes. Destructors were used only where necessary for cleanup.

Challenge 6: File Handling Format Issues

While saving and loading data using `fstream`, formatting inconsistencies caused unreadable or misaligned output in text files.

Solution:

Consistent formatting rules were applied using clear separators, newline handling, and structured output layout. The `.txt` report format was standardized for all modules.

Testing: Screenshots of all modules outputs

Module-1

```
Name : Rida
CNIC : 36104
Age : 19
Contact : 0317745
Roll Number : 061
Semester : 2
GPA : 3.8
Enrolled Courses : 4 0 0 0 0
Grade : A

Name : Hajra
CNIC : 45267
Age : 35
Contact : 0313245
Employee ID : 157
Department : CS
Designation : Lecturer
Assigned Courses : 2

Name : Fatima
CNIC : 56791
Age : 23
Contact : 0315234
Staff ID : 121
Role : Assistant
Salary : 40000

-----
Process exited after 0.2257 seconds with return value 0
Press any key to continue . . .
```

Module-2

```
C:\Users\DELL\Desktop\coding\Module_02.exe
Courses are equal.
Course capacity exceeded!

Merged Waiting List:
Sana
Fatima

Course Information
Course Code: CS101
Course Name: Programming
Credit Hours: 3
Instructor: Dr Ahmed
Capacity: 2/2

Enrollment Details
Student: Hafsa
Course: Programming
Date: 01-Sep-2025
Grade: A

-----
Process exited after 0.3073 seconds with return value 0
Press any key to continue . . .
```

Module-3

LIBRARY CATALOG

Item ID is: 101

The title of book is: Object Oriented Programming

The author of book is: Bjarne Stroustrup

The book was published in the year: 2018

ISBN is: 9780136152504

Genre of the book is: Programming

The copies available are: 5

Item ID is: 102

The title of book is: Data Structures

The author of book is: Mark Weiss

The book was published in the year: 2020

ISBN is: 9781292024475

Genre of the book is: Computer Science

The copies available are: 3

The Item ID: 201

The title is: Artificial Intelligence Review

Author is: John Smith

Publication Year is: 2024

ISSN of journal is: ISSN-56789

Volume: 10

Issue Number is: 2

SEARCH BY TITLE

Item ID is: 102

The title of book is: Data Structures

The author of book is: Mark Weiss

The book was published in the year: 2020

ISBN is: 9781292024475

Genre of the book is: Computer Science

The copies available are: 3

ISSUE ITEM

Book issued successfully.

SAVE CATALOG

LOAD CATALOG

Catalog is loaded successfully.

RETURN ITEM

Overdue item returned.

Fine Amount: Rs. 200

Module-4

```
Student Ref : 25-CS-061
Semester Fee : 90000
Hostel Fee : 30000
Library Fine : 500
Total Paid : 2000
Balance : 122500
Student Ref : 25-CS-390
Semester Fee : 70000
Hostel Fee : 30000
Library Fine : 200
Total Paid : 4000
Balance : 104200
Copy Constructor
Student Ref : 25-CS-061
Semester Fee : 90000
Hostel Fee : 30000
Library Fine : 500
Total Paid : 0
Balance : 120500
Copy Assignment Operator
Student Ref : 25-CS-390
Semester Fee : 70000
Hostel Fee : 30000
Library Fine : 200
Total Paid : 4000
Balance : 104200
Enter Item 1: toy
Enter Item 2: chair
Enter Item 3: bag
Invoice ID : 1
Date : 1-June-2024
Items:
toy
chair
bag
Total Amount : 6400
Enter Item 1: table
Enter Item 2: dress
Invoice ID : 2
Date : 10-Oct-2022
Items:
table
dress
Total Amount : 7800
Total Invoices = 2
Invoice 2 DestroyedInvoice 1 Destroyed
-----
Process exited after 73.66 seconds with return value 0
Press any key to continue . . .
```

Module-5

```
C:\Users\DELL\Desktop\coding\Module-5.exe
Fatima allocated to Room 102
Hafsa allocated to Room 102
Room is full!
Rida allocated to Room 201
HOSTEL REPORT
Block Name: A Block
Room No: 101 Type: Single Floor: 1 Occupancy: 0/1
Room No: 102 Type: Double Floor: 1 Occupancy: 2/2
Student: Fatima
Student: Hafsa
Room No: 201 Type: Triple Floor: 2 Occupancy: 1/3
Student: Rida
Room No: 202 Type: Double Floor: 2 Occupancy: 0/2
Room No: 301 Type: Single Floor: 3 Occupancy: 0/1
Fatima vacated Room 102
HOSTEL REPORT
Block Name: A Block
Room No: 101 Type: Single Floor: 1 Occupancy: 0/1
Room No: 102 Type: Double Floor: 1 Occupancy: 1/2
Student: Hafsa
Room No: 201 Type: Triple Floor: 2 Occupancy: 1/3
Student: Rida
Room No: 202 Type: Double Floor: 2 Occupancy: 0/2
Room No: 301 Type: Single Floor: 3 Occupancy: 0/1
-----
Process exited after 0.2376 seconds with return value 0
Press any key to continue . . .
```

Module-6

```
Fatima allocated to Room 102
Hafsa allocated to Room 102
Room is full.
Rida allocated to Room 201

-----
HOSTEL REPORT
-----

Hostel Occupancy Report

Block: A Block
Room: 101 Type: Single Floor: 1 Occupants: 0/1
Room: 102 Type: Double Floor: 1 Occupants: 2/2
- Fatima
- Hafsa
Room: 201 Type: Triple Floor: 2 Occupants: 1/3
- Rida
Room: 202 Type: Double Floor: 2 Occupants: 0/2
Room: 301 Type: Single Floor: 3 Occupants: 0/1

-----
SEARCH STUDENT
-----
Hafsa is in Room 102

-----
SEARCH ROOM
-----
Room: 201 Type: Triple Floor: 2 Occupants: 1/3
- Rida

-----
VACANT ROOMS
-----
Room 101 has vacancy.
Room 201 has vacancy.
Room 202 has vacancy.
Room 301 has vacancy.
Hostel report saved successfully.
Fatima vacated Room 102

-----
HOSTEL REPORT
-----

Hostel Occupancy Report

Block: A Block
Room: 101 Type: Single Floor: 1 Occupants: 0/1
Room: 102 Type: Double Floor: 1 Occupants: 1/2
- Hafsa
Room: 201 Type: Triple Floor: 2 Occupants: 1/3
- Rida
Room: 202 Type: Double Floor: 2 Occupants: 0/2
Room: 301 Type: Single Floor: 3 Occupants: 0/1
```

Conclusion

This project provided a comprehensive understanding of how Object-Oriented Programming concepts can be applied to build a realistic and modular software system. Instead of studying concepts such as inheritance, polymorphism, abstraction, encapsulation, operator overloading, file handling, and exception handling in isolation, this project made it possible to implement them in the

context of a complete **Campus Management System**. By dividing the system into separate modules such as **Person Management, Course and Enrollment Management, Library System, Hostel Management, and Reporting & Utilities**, the project demonstrated how large applications can be organized into manageable and logically connected components.

One of the most valuable outcomes of this project was learning how to design class hierarchies and relationships in a way that reflects real-world entities. For example, the use of an abstract `Person` base class with derived classes such as `Student`, `Faculty`, and `Staff` showed how common attributes and behaviors can be shared while still allowing specialization in child classes. Similarly, the library and hostel modules helped in understanding abstract classes, multiple inheritance, composition, aggregation, and the use of pointers to model relationships between objects. Implementing custom exceptions, operator overloading, and file I/O also strengthened practical programming skills beyond basic class design.

Another important lesson from this project was the importance of modularity and planning in software development. Since the project consisted of multiple modules, each one had to be designed in a way that could work independently while still fitting into the larger system. This required careful thinking about responsibilities of classes, how objects interact, and how data should be stored and accessed. It also highlighted common programming challenges such as managing pointers, handling file operations correctly, and ensuring that code remains readable and maintainable as the project grows.

Overall, this project was a valuable learning experience because it connected theoretical OOP concepts with practical implementation. It improved not only my understanding of C++ syntax and programming techniques, but also my ability to think in terms of system design, modular development, and problem solving. As a result, this project served as a strong foundation for understanding how object-oriented principles can be used to build larger and more structured software systems in real-world scenarios.

References

- YouTube channels
- <https://cplusplus.com/doc/>
- <https://www.geeksforgeeks.org/c-plus-plus/>